GitHub as an Almanacic Zine Host: Repositories as Living Public Scholarship

2026-07-06

GitHub as an Almanacic Zine Host: Repositories as Living Public Scholarship

Version: v0.001
Date: 2026-07-06
Status: working paper

Abstract

A GitHub repository is usually treated as a container for software or as backstage infrastructure for a finished publication. This paper argues for a second use: the repository as an almanacic zine host. In this mode, the repository is not only where a public object is stored. It is part of the object. The README works as a cover and route card. Folders work as rooms. Issues work as letters, errata, source challenges, and repair tickets. Branches work as alternate editions. Pull requests work as staged repair. Releases and tags work as numbered issues. Permanent links pin a cited state. A reader-facing site, built from the same repository, gives a front surface without cutting the source trail away.

The paper separates metaphor from method. Calling a commit "marginalia" is not enough. The method test is whether repository mechanics change what a reader may cite, what a contributor may repair, what an owner may govern, and what a future reader may re-open. The paper gives a feature map, a worked example, a custody checklist, and a failure taxonomy for using GitHub as living public scholarship.

1. Object

The object is a publication form:

> A repo-zine is a public knowledge object whose files, route map, versions, repairs, questions, and reader participation live in one repository.

It is not the claim that GitHub was designed for zines. It is not the claim that every paper should become a repository. It is the claim that some scholarly and public objects lose working surface when they are flattened into one PDF, one blog post, or one frozen web page.

The class of objects is familiar:

- living bibliographies
- public methods
- teaching packs
- field notebooks
- community archive packets
- conlang grammars
- errata ledgers
- data dictionaries
- local history almanacs
- multi-version public essays
- paper families with source maps

These objects are not only text. They have route, recurrence, source state, dispute state, reader letters, and repairs. A repo-zine hosts those jobs in the same place as the readable object.

2. H0

H0: GitHub is only code storage, software collaboration, issue tracking, and static site hosting. The zine and almanac reading is decorative.

Working claim: H0 loses power when ordinary GitHub mechanics do publication work that a flat artifact does not do as cheaply.

The paper tests H0 by asking whether the repository mechanics perform decisions:

Mechanic	Publication job
README	front door, status card, route card
folders	rooms, shelves, evidence zones
commits	dated change record
issues	letters, errata, source challenges, open questions
pull requests	staged repair before merge
branches	alternate editions and experiments
tags	named state markers
releases	numbered issues with assets and notes
permalinks	exact cited file state
wiki	long-form documentation space
Pages	reader-facing surface built from the same source

If these mechanics only provide style, H0 survives. If they change citation, repair, governance, and reading route, H0 does not.

3. Source base

The platform facts in this paper use GitHub documentation. GitHub's README documentation says a README tells visitors what a project does, why it matters, how to start, where to get help, and who maintains it.[1] GitHub Issues documentation describes issues as a way to plan, discuss, and track work, including reports, ideas, and anything a team needs to write down or discuss.[2] GitHub documentation describes forks as independent repositories that keep relation to an upstream repository, with their own branches, issues, pull requests, tags, labels, and wikis.[3]

GitHub wiki documentation says a repository wiki may host long-form project documentation, and that wiki pages may be edited on GitHub or locally through a Git workflow.[4][5] GitHub release documentation states that releases are based on tags and may include notes, links, and assets.[6] GitHub permanent-link documentation says ordinary branch URLs may move with new commits, while commit-based links pin the exact version seen by the reader.[7] GitHub Pages documentation describes hosting a website from a repository.[8]

The zine and archive side of the paper uses scholarship on zines as self-published community archive practice and as a form for visualization and public expression.[9][10] The citation side is aligned with software and data citation work that treats versioned digital objects as scholarly objects needing identification, attribution, and access.[11][12]

4. Repo-zine grammar

A repo-zine needs a grammar because the same platform feature may be either decoration or method.

Repo part	Zine or almanac role	Method test
`README.md`	cover, masthead, reader route	A new reader learns object, status, and first path.
`START_HERE.md`	route card	The reader does not have to infer the entry path from the file tree.
`/issues/`	numbered essays or seasonal issues	Items have dates, version IDs, and intended readers.
GitHub Issues	letters, corrections, open questions	Reader input changes repair state or route state.

Repo part	Zine or almanac role	Method test
Pull requests	staged repair	Proposed edits are reviewed before becoming the main edition.
Branches	alternate editions	Experiments stay available without replacing the front edition.
Tags	named moments	A version can be cited without guessing.
Releases	public issue drops	A package is frozen with notes and assets.
Permanent links	citation pins	A claim points to the same file state later.
`/evidence/`	source shelf	Evidence is separated from style and stickers.
`/errata/`	repair ledger	Changes do not vanish after correction.
`/docs/` or Pages	reader surface	The readable site stays near the source.

This grammar rejects the PDF-only model without rejecting PDFs. A PDF remains a reader edition. The repo holds the work surface around it.

5. Metaphor versus method

A repo-zine fails when it only renames ordinary files with cute labels. It passes when the repository mechanics change available action.

Metaphor only

```
README = cover
but it does not tell the reader status, route, version, or owner.

Issues = letters
but reader replies never enter the repair ledger.

Releases = zine issues
but the release is only a download with no version note.

Branches = alternate timelines
but no branch has a named job or merge rule.
```

Method

```
README = cover
because it names the object, route, current status, citation form, and maintainer.

Issue = letter
because it can be labeled as erratum, source challenge, reader note, or open question.

Release = issue
because it freezes a numbered edition with assets, date, notes, and source state.

Branch = alternate edition
because it holds an experiment with a route back to main, archive, or rejection.
```

The method is behavioural. It is not aesthetic. The test is what the repo lets people do.

6. Almanac logic

An almanac is not just a list. It is a recurring public table of seasonal, dated, practical knowledge. A repo-zine borrows that recurrence:

```
monthly issue
seasonal source map
annual repair ledger
dated route note
versioned glossary
open questions carried forward
reader letters grouped by issue
```

This gives living scholarship a rhythm. The object may change, but the change arrives through named drops, not through silent replacement.

A repository makes this pattern cheap:

- tags name the moment
- releases package the issue
- commits show the path
- issues carry unresolved items
- branches hold experiments
- Pages presents the issue to nontechnical readers

The almanac form matters because many public knowledge objects recur. They are not finished once. They come back.

7. Worked example: river-town almanac

Consider a public local-knowledge repo:

```
river-town-almanac/
  README.md
  START_HERE.md
  STATUS.md
  CITATION.cff
  CHANGELOG.md
  /issues/
    2026-07-summer-water.md
    2026-10-autumn-wind.md
  /rooms/
    river.md
    garden.md
    birds.md
    recipes.md
    folklore.md
  /evidence/
    source_map.md
    water_log.md
    oral_history_register.md
  /errata/
    open.md
    repaired.md
  /.github/
    ISSUE_TEMPLATE/
      erratum.yml
      field-note.yml
      reader-letter.yml
      source-challenge.yml
  /docs/
    index.md
```

Here the repository is not a storage bin. It does work.

`README.md` gives the reader the object and route. `START_HERE.md` gives the first path for teachers, residents, and contributors. `/issues/2026-07-summer-water.md` is a seasonal issue. `/evidence/source_map.md` states what each claim depends on. `/errata/open.md` keeps unsettled items visible. A release named `v2026.07` freezes the July issue. A permanent link to `water_log.md` pins the evidence state used in a paragraph. A branch named `autumn-redesign` can hold a new layout while the main edition remains stable.

This is method, not metaphor, because the platform decides how reading, repair, and citation happen.

8. Custody rules

A repo-zine needs custody rules so the object does not become a maze.

Rule 1: front door first

The README must answer:

```
What is this?
Who is it for?
What is current?
Where should a new reader start?
How should this be cited?
How should errors be reported?
What is not inside this repo?
```

Rule 2: folders need jobs

Folders should not be named only for texture. Each folder should have a job and a `README.md` or index note.

```
/evidence/ = source shelf
/errata/ = repair ledger
/issues/ = public editions
/rooms/ = topic spaces
/docs/ = reader-facing site source
/archive/ = old editions kept for custody
```

Rule 3: releases are issues

A release should say what it freezes, what changed, what remains open, and which assets belong to the issue.

Rule 4: questions stay visible

Open questions should not be hidden because they are unfinished. They should be marked with status and route.

Rule 5: source and decoration separate

A sticker, image, joke, or room label may help readers. It should not masquerade as evidence. Evidence belongs in the source shelf.

Rule 6: cite state, not only location

Branch links may point to a later file state. Claims that depend on exact text should cite a tag, release, or commit permanent link.

Rule 7: forks are editions

A fork may be a translation, local edition, classroom edition, or disputed branch. It is not only a technical copy.

9. Failure taxonomy

Maze repo

The repository has many files but no first path. A reader must infer the object from filenames. Repair: add `README.md`, `START_HERE.md`, and per-folder route notes.

Dead workshop

The repository advertises living repair but has no recent status note, no release path, and no owner route. Repair: add `STATUS.md`, an issue policy, and an archive or active label.

Source cosplay

The repository displays citations, badges, or source-like folders without tying claims to source state. Repair: add a source map with claim, source, state, and status.

Sticker takeover

Local language, jokes, mascots, and rooms become the object rather than the route. Repair: separate wall material from evidence and route material.

Platform dependence

The object only works inside one platform interface. Repair: keep exportable Markdown, a PDF edition, a citation file, and a release package.

Infinite draft

The object never publishes because the repo is always being rearranged. Repair: issue named releases at fixed intervals.

Silent replacement

The front text changes without a reader-facing change note. Repair: use changelog, release notes, and commit messages that name object-level changes.

10. Academic use

Repo-zines fit scholarly situations where the finished object and the route to it both matter.

Public methods

A methods paper often hides failed branches, source arguments, and repair choices. A repo-zine can publish the method paper, the protocol, the source map, the errata ledger, and the issue history as one object with multiple surfaces.

Living bibliographies

A bibliography changes as the field changes. A repo-zine can release dated bibliography issues, accept source challenges through issue templates, and keep rejected additions visible with reasons.

Teaching packs

A teacher may need a stable handout and an active repair surface. Releases provide class editions. Issues carry student questions and errata. Pages gives a reader surface.

Community archive packets

Community archive work may need shared stewardship, local language, photographs, oral histories, annotations, and contestation. Zine scholarship already treats zine-making as both product and process.[9] A repository can make the process visible without forcing all readers into the full archive.

Nonlinear essays

Some arguments have rooms, not chapters. A repo-zine lets a reader choose the front essay, the source shelf, the glossary, the errata room, or the discussion wall.

11. Relation to existing scholarly infrastructure

The repo-zine does not replace journals, archives, DOI registries, institutional repositories, or data repositories. It adds a work surface before, beside, and after them.

A journal article may be the sanctioned edition. A repository may hold:

```
source maps
version notes
review replies
supplements
teaching editions
errata
reader letters
public protocols
```

The repo-zine is strongest when paired with archival deposits and citation records. GitHub is not a preservation guarantee by itself. The artifact should have export paths, tags, release packages, and, where suitable, external archiving.

12. Minimal template

A small repo-zine does not need elaborate infrastructure. It needs route.

```
repo-zine/
  README.md
  START_HERE.md
  STATUS.md
  CITATION.cff
  CHANGELOG.md
  /issues/
    issue-001.md
  /evidence/
    source_map.md
  /errata/
    open.md
    repaired.md
  /docs/
    index.md
```

Minimal README sections:

```
Object
Audience
Current edition
Start path
Citation
How to report an error
Folder map
Maintenance status
License
```

Minimal issue template labels:

```
erratum
source challenge
reader letter
field note
route problem
edition proposal
```

Minimal release note:

```
Edition
```

```
Date
Files included
What changed
Open questions
Citation
Known export locations
```

13. Discussion

The repo-zine model matters because it treats route as publication material. Many scholarly artifacts already have route, but route is hidden in private drafts, email, notebooks, commit history, chat transcripts, reviewer exchanges, and file names. A repo-zine does not expose everything. It exposes the parts that help readers understand state, repair, and citation.

This is a governance model as much as a format. It asks authors to state what may be changed, what must be frozen, what counts as a repair, how reader input enters, and how a future reader reaches the cited state.

The main discipline is smallness. A repo-zine should not eat every note. It should keep the public object, its route, and its repair ledger near each other. Cold archive material may remain outside the active surface with an index.

14. Conclusion

GitHub is underused as a host for living public scholarship because it is read too narrowly. Its ordinary machinery already carries many jobs of zines and almanacs: self-publication, recurrence, reader letters, marginal repair, alternate editions, source pinning, and issue-based release.

The repo-zine is not a brand. It is a method for objects that need a front edition and a work surface. The workshop can be part of the publication, provided that the route is named, the source shelf pays rent, and the cited state can be reached again.

Local source packet

This v0.001 was drafted from the uploaded paper bundle and desk-note intake using these local anchors:

- `extra_extra_expanded/NAK_github_as_non_linear_almanacic_zine_host_v0_001.md`
- `extra_extra_expanded/github_zine_b_pass_v0_002_bundle__unzipped/github_zine_almanac_B_c`
- `extra_extra_expanded/github_zine_b_pass_v0_002_bundle__unzipped/github_zine_almanac_B_s`
- `extra_extra_paper_strike_update.md`

References

[1] GitHub Docs. “About the repository README file.” https://docs.github.com/en/repositories/managing-

your-repositorys-settings-and-features/customizing-your-repository/about-readmes

[2] GitHub Docs. "About issues." https://docs.github.com/articles/about-issues

[3] GitHub Docs. "About forks." https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/working-with-forks/about-forks

[4] GitHub Docs. "About wikis." https://docs.github.com/en/communities/documenting-your-project-with-wikis/about-wikis

[5] GitHub Docs. "Adding or editing wiki pages." https://docs.github.com/en/communities/documenting-your-project-with-wikis/adding-or-editing-wiki-pages

[6] GitHub Docs. "Managing releases in a repository." https://docs.github.com/en/repositories/releasing-projects-on-github/managing-releases-in-a-repository

[7] GitHub Docs. "Getting permanent links to files." https://docs.github.com/en/repositories/working-with-files/using-files/getting-permanent-links-to-files

[8] GitHub Docs. "What is GitHub Pages?" https://docs.github.com/en/pages/getting-started-with-github-pages/what-is-github-pages

[9] Baker, S., and Cantillon, Z. "Zines as community archive." *Archival Science* 22, 539-561, 2022. https://doi.org/10.1007/s10502-022-09388-1

[10] McNutt, A. "On the Potential of Zines as a Medium for Visualization." arXiv:2108.02177, 2021. https://arxiv.org/abs/2108.02177

[11] Smith, A. M., Katz, D. S., Niemeyer, K. E., and FORCE11 Software Citation Working Group. "Software citation principles." *PeerJ Computer Science* 2:e86, 2016. https://doi.org/10.7717/peerj-cs.86

[12] Wilkinson, M. D., Dumontier, M., Aalbersberg, I. J. J., et al. "The FAIR Guiding Principles for scientific data management and stewardship." *Scientific Data* 3, 160018, 2016. https://doi.org/10.1038/sdata.2016.18